

Today's Agenda:-

1. Print Paths in Staircase
2. Print all maze paths.
3. Subset Sum Equal to K.

Starting 7:05

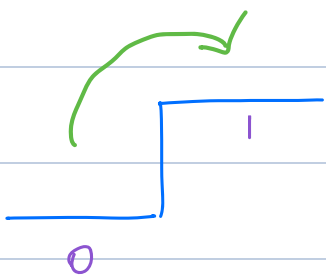
1. Print all paths in a staircase

You are climbing a staircase and it takes **A** steps to reach the top.

Each time you can either climb **1** or **2** steps. In how many **distinct ways** can you climb to the top?

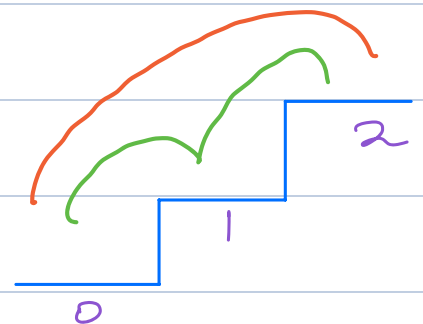
You need to return all the **distinct ways** to climb to the top in **lexicographical order**.

ex $N = 1$



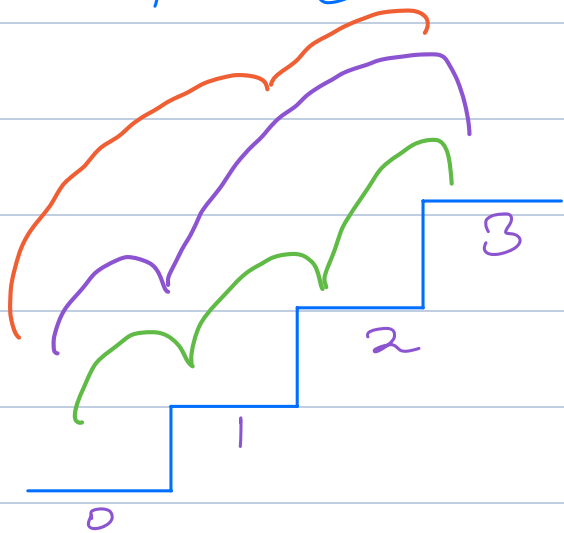
o/p:- 1

ex'r $N = 2$



o/p:- {1, 1}
{2}

ex:- $N = 3$

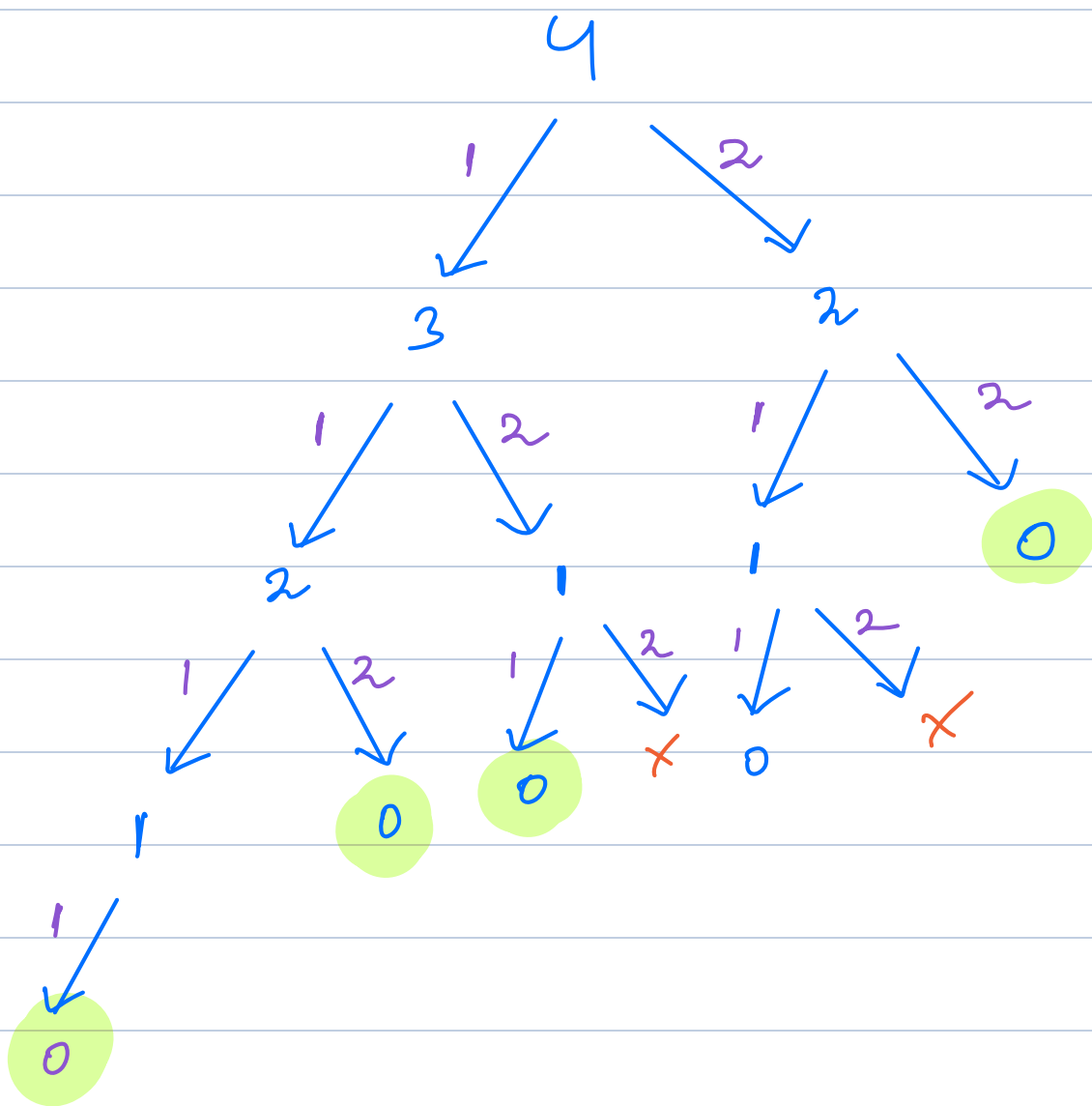


o/p:- $\{1, 1, 1\}$
 $\{1, 2\}$
 $\{2, 1\}$

Idea:- In order to reach the N^{th} step, there are 2 ways:

1. Take a jump of 1 from the $N-1^{\text{th}}$ step.

2. Take a jump of 2 from $N-2^{\text{th}}$ step.



Maintain a curr. array in each iteration.
Add the step you're taking to the curr. array
before making a recursive call & then
remove the step added for further calls:

void steps ($\overset{\nearrow N}{N}$, list < int > curr) { $\nearrow 13$

if ($N == 0$) { print (curr); return; }

curr.add(1);
steps ($N-1$, curr);
curr.removeLast();

if ($N \geq 2$) {
curr.add(2);
steps ($N-2$, curr);
curr.removeLast();
}

3

T.C $\rightarrow O(2^N)$.

S.C $\rightarrow O(\underline{N})$.

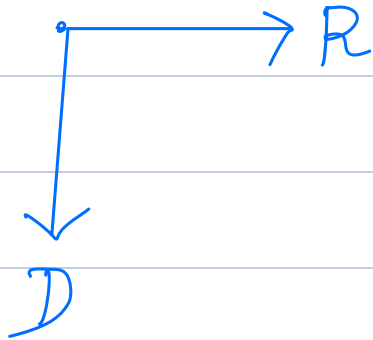
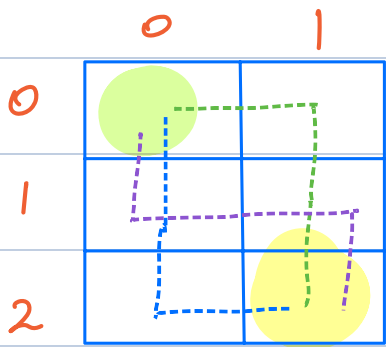
2. Print all maze paths.

You are given the dimensions of a rectangular board of size **A x B**. You need to print all the possible paths from **top-left** corner to **bottom-right** corner of the board.

You can only move down (denoted by 'D') or right (denoted by 'R') at any point in time.

NOTE : You need to return all the paths in **Lexicographically sorted order**. A lexicographic order is an arrangement of characters in alphabetical order.

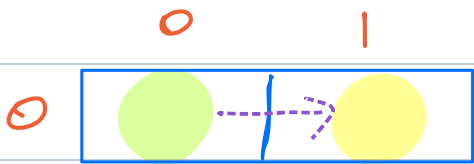
ex:- 3 x 2



o/p:- "DDR"
"DRD"
"RDP"

"RDD" ③
"DDR" ①
"DRD" ②

ex) 1 x 2



o/p:- "R"

Idea:- keep moving in both the direction & keep track of the move you made in a string variable.

Actively, avoid making a move that goes out of bound of the matrix.

To maintain a lexicographical order, i.e. (D before R); always go in the downward direction first; then go rightwards.

N, m

void allPaths(i, j , path) {

if ($i == N-1$ && $j == m-1$) { print(path); return; }

if ($i+1 \leq N-1$) {

allPaths($i+1, j$, path + "D");

if ($j+1 \leq m-1$) {

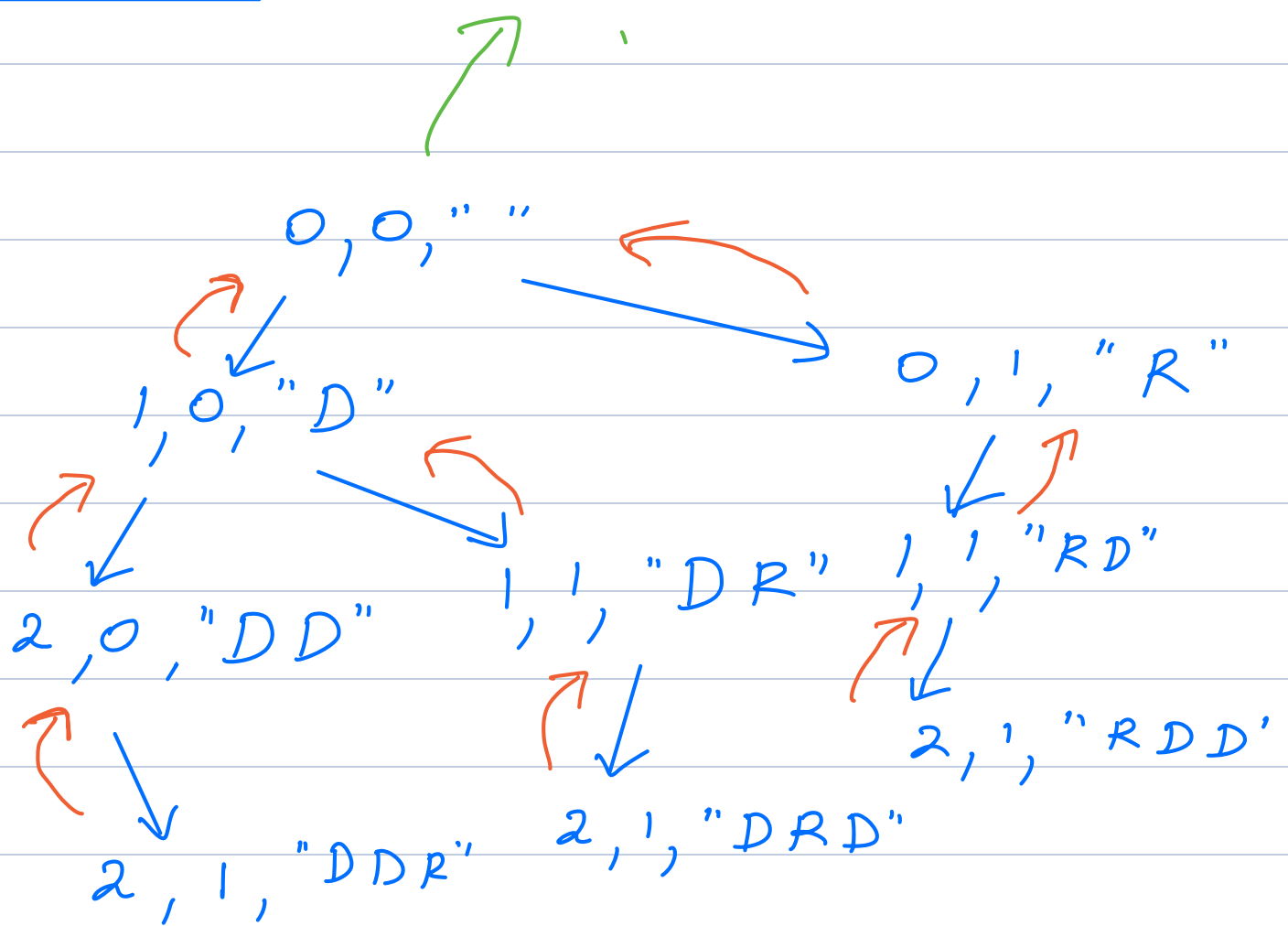
allPaths($i, j+1$, path + "R");

}

Dry Run:-

$N=3; m=2$

	0	1
0	-	-
1		
2		



o/p:- "DDR"
"DRD"
"RDD"

T.C $\rightarrow O(2^{N+m})$
S.C $\rightarrow O(N+m)$
=

3. Subset Sum equals K .

Given an integer array A of size N .

You are also given an integer B , you need to find whether there exist a subset in A whose sum equal B .

If there exist a subset then return 1 else return 0 .

Note : Sum of elements of an empty subset is 0 .

ex:- $\{ \overset{0}{3}, \overset{1}{34}, \overset{2}{-3}, \overset{3}{5}, \overset{4}{12}, 2 \}$
 $K = 9$ 1.

ex:- $\{-8, 34, 4, 0, -5, -2\}$
 $K = -20$ 0.

Idea:- Generate all possible subsets by selecting or rejecting an element at idx.

Maintain a variable $curSum$. If rejecting, add 0 ; if selecting, add $arr[idx]$ to the $curSum$.

When covered all indices, if $curSum$ equals K ; return 1 ; else return 0 ;

int subsetSumK (arr, curSum, idx) {

if (idx == arr.size()) {
 if (curSum == k) { return 1; }
 else { return 0; }

if (subsetSumK (arr, curSum, idx + 1)
 == 1)

//

subsetSumK (arr, curSum
 + arr[idx], idx + 1);

== 1) {

return 1;

else {

return 0;

T.C $\rightarrow O(2^N)$

S.C $\rightarrow O(N)$